

Bash 셸 기초

Bash Basics

- 셸과 Bash
- 명령어 사용 기초
- 편리한 셸 기능

셸과 BASH

□ Shell

Shell이란?

Shell은 사용자가 운영체제의 핵심인 커널(Kernel)과 상호작용할 수 있도록 해주는 인터페이스예요.

쉽게 말해, 사용자가 명령을 입력하면 Shell이 그것을 해석해 커널에 전달하고, 결과를 다시 사용자에게 보여주는 역할을 합니다.

Shell은 OS에만 있을까?

대부분의 Shell은 OS 환경에서 동작하지만, 응용 프로그램이나 개발 환경에서도 Shell과 유사한 인터페이스가 존재해요.

예: Python의 REPL, 데이터베이스의 SQL 콘솔, 게임 개발 엔진의 명령 콘솔 등도 일종의 Shell로 볼 수 있습니다.

셸의 종류 : CLI, GUI, SHELL

CLI 셸의 종류

Bourne shell
(sh)

C shell
(csh)

Korn shell
(ksh)

Bourne-Again
shell
(bash)

Z shell
(zsh)

PowerShell

뭐 더 좋은 거
없을까?

□ Bash (Bourne Again Shell)

- GNU 표준 셸 → 리눅스 표준 셸
- 1989년 처음 배포

□ Bash 정보

- 다운로드: <http://www.gnu.org/software/bash>
 - bash가 설치되어 있지 않은 경우 또는 upgrade
- 매뉴얼
 - 온라인: <http://www.gnu.org/software/bash/manual>
 - 명령행에서 help 명령으로 도움말 기능 제공



□ 셸의 공통 기능

- 다수의 내장(built-in) 명령어 제공
- 메타문자
- 입력/출력/오류의 재지정 (redirection)
- 파이프라인
- 명령어 대치/완성/편집
- 환경 변수와 지역변수
- 후면(background) 처리와 작업(프로세스) 제어
- 셸 프로그래밍



셸 실행 환경

□ CLI 셸 실행 장치 – 터미널 (terminal)

- 사용자로부터 텍스트 입력
→ 실행 결과를 텍스트로 출력



□ GUI 환경에서 명령어 실행 환경

- 가상 터미널 프로그램: gnome-terminal, xterm, PowerShell, ...
 - 키보드 외에 마우스도 활용할 수 있음
- ssh clients for networked environments: PuTTY 등

이름이 왜 terminal일까?

지금 알 필요는 없지만...
console, tty, pty, pts

셸 시작과 종료

□ 셸의 시작

- 터미널을 실행하거나 CLI 환경에서 로그인을 하면 자동으로 기본 셸(=bash) 시작
- 셸 프로그램 이름으로 명령 (bash, sh, dash, ...)

□ 셸의 종료

- exit
- logout (로그인했을 때)
- Ctrl + d
 - 입력 끝 표시와 동일하기 때문에 사용하지 않는 편이 좋다.
- 가상 터미널 닫기
 - 터미널 내부의 탭을 비롯한 모든 셸이 동시에 종료됨

- 도움말: man
- 명령어 구조
- 명령어 종류
- 명령어 실행 기록: script

명령어 사용 전에...

도움말 보기: man

□ 특정 명령어나 표준 API에 대한 매뉴얼 출력 명령

- 매뉴얼 페이지(manpage)
- troff/groff로 형식 지정
→ less로 출력
 - 정해진 모양대로 한 페이지씩 출력
 - 다음 페이지: space bar/f
 - 이전 페이지: b
 - 검색: /단어
 - 종료: q

```
MAN(1) Manual pager utils MAN(1)
NAME 이름과 기능 요약
man - an interface to the system reference manuals

SYNOPSIS 사용법
man [man options] [[section] page ...] ...
man -k [apropos options] regexp ...
man -K [man options] [section] term ... 이거 다 해석할 줄 알아야 함
man -f [whatIs options] page ...
man -l [man options] file ...
man -w|-W [man options] page ...

DESCRIPTION 설명
man is the system's manual pager. Each page argument given to man is
normally the name of a program, utility or function. The manual page
associated with each of these arguments is then found and displayed. A
section, if provided, will direct man to look only in that section of
the manual. The default action is to search in all of the available
sections following a pre-defined order (see DEFAULTS), and to show only
the first page found, even if page exists in several sections.

The table below shows the section numbers of the manual followed by the
Manual page man(1) line 1 (press h for help or q to quit)
```

도움말 보기: man

□ man 페이지 구성 항목

- NAME, SYNOPSIS, CONFIGURATION, DESCRIPTION, OPTIONS, EXIT STATUS, RETURN VALUE, ERRORS, ENVIRONMENT, FILES, VERSIONS, CONFORMING TO, NOTES, BUGS, EXAMPLE, AUTHORS, SEE ALSO 등

□ SYNOPSIS 표기법

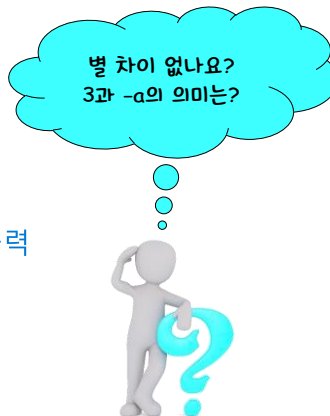
bold text	적힌 그대로 사용
<i>italic text</i>	적절한 인자로 대체해서 사용
[-abc]	[] 내에 있는 하나 또는 모든 인자가 선택적 (여러 개 동시 사용 가능)
-a -b	에 의해 분리된 옵션은 동시 사용 불가 (exclusive OR)
<u>argument</u> ...	argument가 반복 가능
[<u>expression</u>] ...	[] 내의 수식 전체가 반복 가능

도움말 보기: man

□ 섹션의 효과 - 다음을 실행해 보자.

```
$ man printf
$ man 3 printf
$ man -a printf
```

옵션 -a -> 섹션별 모든 메뉴얼 페이지 출력



□ 명령어 구조

- 하나의 줄이 하나의 명령: 명령행(Command line)

```
$ ls -la /usr/bin/a*
```

prompt command options arguments

-l 자세한 정보(권한, 소유자, 크기, 수정시간) 표시
-a 숨김 파일(.으로 시작하는 파일)까지 표시

이 명령을 실행해 보고
결과를 설명해 봅시다.
prompt?
ls는 뭐 하는 명령?
option은 뭐지?
argument는 뭐야...
이상한 글자도 있다. *

□ 내장 명령 (built-in command)

- bash 내에 함수로 구현되어 있는 명령

□ 셸 키워드 (keyword)

- 셸 프로그래밍을 위한 요소

!	[[	]]	{	}	case	do	done	elif	else	esac
fi	for	function	if	in	select	then	time	until	while	

□ 셸 함수명 (function)

□ 실행 가능한 파일의 이름

- 컴파일된 실행 코드 - 리눅스 기본 설치 프로그램(/bin, /usr/bin 등), 사용자 프로그램

□ 별명 (alias)

내장 명령 (built-in command)
셸 자체에 포함된 명령어로, 별도의 실행 파일 없이 셸 내부 함수로 동작합니다. 예: cd, echo, exit

셸 키워드 (keyword)
셸 스크립트를 작성할 때 사용하는 예약어입니다.

예: if, else, fi, for, while, case, do, done
→ 프로그래밍 언어의 문법을 구성하는 핵심 요소라고 보면 됩니다.

함수명 (function)
사용자가 직접 정의해서 재사용할 수 있는 명령 집합입니다.

실행 가능한 파일 이름
/bin, /usr/bin 같은 디렉터리에 있는 프로그램들, 또는 사용자가 만든 실행 파일을 의미합니다. 예: ls, grep, python

별명 (alias)
긴 명령어를 짧게 줄여 쓰기 위해 지정하는 이름입니다. 예:

□ Built-in Commands

- 셸 내부의 함수로 구현
 - 셸 프로세스 내에서 실행
 - 일반 명령어 및 프로그램은 자식 프로세스 생성 후 실행

□ 주요 내장 명령

- help, cd, alias, exit, printf, bg, fg, jobs, history, source, set, ...
- pwd, kill, **test**, **echo**
 - 일반 명령어에도 있으며 기능이 같다.

set만 입력하면 현재 셸에 설정된 모든 변수와 함수 목록을 출력합니다.
지금까지 실행한 명령어 목록이 변수와 함께 출력

백그라운드 실행 중인 작업이나 일시 중지된 작업을 확인

source 파일명 → 해당 파일의 내용을 현재 셸에서 실행

kill [옵션] PID
PID(Process ID): 종료하려는 프로세스의 고유 번호

kill 1234 # PID 1234인 프로세스 종료

□ 셸 도움말

- 셸 내장 명령 및 키워드에 대한 도움말
- help - 전체 명령/키워드 요약
- help 이름 - 이름에 대한 상세 설명
- 옵션
 - -d: 간략한 설명
 - -m: manpage 형식의 출력
 - -s: 사용법만 출력

```

ssluser@ssluser-VirtualBox: ~
ssluser@ssluser-VirtualBox:~$ help
GNU bash, version 5.0.17(1)-release (x86_64-pc-linux-gnu)
셸 명령어는 내부적으로 정의되어 있습니다. 'help'를 입력하면 목록이 보입니다.
'name' 함수에 대해 더 많은 것을 알아보려면 'help name'을 입력하십시오.
일반적인 셸에 대해서 더 많은 것을 알아보려면 'info bash'를 사용하십시오.
목록에 없는 명령어에 대해 더 많은 것을 알아보려면 'man -k' 또는 'info'를 사용하
십시오.

별표(*)가 옆에 있는 명령어는 사용할 수 없음을 의미합니다.

job_spec [&]                                history [-c] [-d offset] [n] or hist>
(( expression ))                            if COMMANDS; then COMMANDS; [ elif C>
. filename [arguments]                      jobs [-lnprs] [jobspec ...] or jobs >
:                                             kill [-s sigspec | -n signum | -sigs>
[ arg... ]                                  let arg [arg ...]
[[ expression ]]                           local [option] name[=value] ...
alias [-p] [name[=value] ... ]             logout [n]
bg [job_spec ...]                          mapfile [-d delim] [-n count] [-o or>
bind [-lpsvPSVX] [-m keymap] [-f file>    popd [-n] [+N | -N]
break [n]                                  printf [-v var] format [arguments]
builtin [shell-builtin [arg ...]]          pushd [-n] [+N | -N | dir]
caller [expr]                               pwd [-LP]
case WORD in [PATTERN] [PATTERN]...>      read [-ers] [-a array] [-d delim] [->
cd [-L|-P] [-e] [-@]] [dir]               readarray [-d delim] [-n count] [-o >
    
```

명령어 실행 기록: script

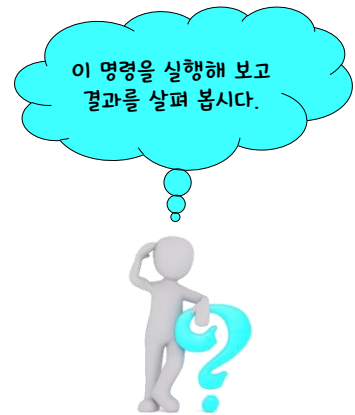
□ 현재 실행 중인 내용을 그대로 기록

```
script [-a] [filename]
```

- -a: 기존 파일에 덧붙임
- filename: 이름이 없으면 'typescript'로 만들어짐
- 끝낼 때는 exit

□ 주의! – 이미 만들어진 파일이 있을 경우

- 반드시 "-a" 옵션을 사용하거나
- 새 파일 이름을 지정하여 만들 것.



- 컨트롤 키 ctrl
 - 내용 자동 완성 tab
 - 히스토리(history) -> 큐 자료구조 방식임
 - 표준 입출력과 입출력 재지정(redirection)
- script
help pwd man printf
ls
history
- stdin stdout stderr
> 출력
< 입력
>> 기존 파일에 출력 추가
- ## 편리한 셸 기능